

HƯỚNG DẪN & HIỆN TRẠNG KỸ THUẬT SMART PARKING

ARM KIT 01 • Snapshot project hiện tại • 07/09/2026

Vai trò tài liệu: tài liệu bàn giao kỹ thuật cho người/AI tiếp tục phát triển project. Phải đọc tài liệu này cùng source hiện tại; không coi các tính năng “dự kiến” là đã triển khai.

1. Cấu trúc project cần biết

Các khu vực quan trọng:

Core/Inc/ → header firmware
Core/Src/ → source STM32
Core/Python/ → PC backend/client/database
Core/Python/ANPR/ → camera, YOLO, OCR, ANPR service
Core/Python/web/ → Flask Dashboard
Drivers/ → CMSIS + STM32F2 HAL
parking_iot.ioc → cấu hình STM32CubeMX
SmartParking.exe → bản build launcher hiện có

Không nên chỉnh sửa các file build/debug để giải quyết lỗi source. Khi sửa firmware, ưu tiên Core/Src, Core/Inc và parking_iot.ioc; khi sửa PC/ANPR/Web, ưu tiên Core/Python.

2. Phần cứng ARM KIT - pin map dùng để phát triển

Nhóm	Pin	Hiện trạng
4 slot	PE13, PE12, PE11, PE10	DI.0-DI.3 theo sơ đồ kit
Gate IN/OUT mô phỏng	PE9, PE8	DI.4/DI.5 theo thiết kế; source hiện có GATE_IN/GATE_OUT
RS232	PA9, PA10	USART1; PC communication
RS485_1	PB10, PB11, PE15	TX/RX + DE/RE; ứng viên chính cho RFID RS485
RS485_2	PC10, PC11, PA3	TX/RX + DE/RE
EEPROM 24C16	PC12, PA15	SDA/SCL software I2C
74HC595/165	PB5, PB4, PB3, PE0, PE1, PE2	SPI.3 + latch/load/OE
LCD	PD0-PD7	D4-D7, RS/RW/E và backlight
Buzzer	PE3	Còi, active high
LED	PC6, PD15...PD10, PB12	LED1-LED8
DO	PA8, PC9, PC8/PB6, PC7/PB7, PC6, PD15...PD10, PB12	Cần chọn riêng output cho barrier

Theo tài liệu ARM KIT, RS485_1 là PB10/PB11 với DE/RE tại PE15; RS232 là PA9/PA10. [filecite\turn13file0\L3-L4](#)

3. Cấu hình .ioc hiện tại - điểm cần nhớ

File parking_iot.ioc hiện chỉ khai báo TIM1, USART1 và GPIO; không có USART3/USART4 hoặc driver RFID trong cấu hình hiện tại. PE8/PE9 và PE10-PE13 đang được cấu hình GPIO output trong snapshot. Vì vậy không được kết luận rằng các chân này đã sẵn sàng cho cảm biến thật chỉ dựa trên tên DI/GATE.

Khi triển khai RFID RS485, cần cập nhật .ioc để kích hoạt đúng peripheral UART/USART phù hợp, cấu hình GPIO DE/RE nếu cần và kiểm tra xung đột với chức năng hiện tại trước khi sinh lại code.

4. Firmware: luồng xử lý

main.c hiện khởi tạo HAL → clock → GPIO → TIM1 → USART1 → LCD → EEPROM → app_parking → RS232, sau đó vòng lặp gọi app_parking_process() và rs232_process().

Luồng logic parking:

Input slot/gate → parking_slot → RAW/STAB → app_parking → parking_gate → UI/storage → RS232 status

Luồng gate phần mềm:

GATE event → kiểm tra được phép → OPEN_GATE_IN/OUT → WAIT_PARK hoặc WAIT_EXIT → chờ slot transition → completed event

5. Gate hiện tại và cách thay nút bằng barrier sau này

Hiện tại parking_gate.c đọc GATE_IN/GATE_OUT như các nút và có debounce/edge event. Các hàm parking_gate_open_entry() và parking_gate_open_exit() mới điều khiển cờ trạng thái phần mềm, chưa điều khiển motor.

Thiết kế nên tách thành 3 lớp:

Lớp	Nhiệm vụ
Gate command	STM32 quyết định mở IN/OUT
Motor driver	Nhận tín hiệu DO từ STM32 và cấp công suất cho motor/servo; không nối motor trực tiếp vào MCU
Gate feedback	Cảm biến mở/đóng hoặc hành trình; dùng GPIO input riêng

Không dùng lại cùng một GPIO vừa làm input cảm biến vừa làm output motor.

6. RS232 / Python / SQLite

Firmware RS232 dùng USART1 115200 8N1. PC Client mở COM theo config.py, hỏi GET_STATUS định kỳ và lưu trạng thái vào Core/Python/parking_history.db. Web Dashboard cũng đọc chính database này.

Database hiện có:

Bảng	Vai trò
status_history	Snapshot trạng thái 4 slot, FREE/OCC, IN/OUT, gate
parking_events	Sự kiện parking; có trường plate
current_parking	Xe đang trong bãi: plate, slot, entry_time, exit_time
anpr_pending	Biển số ANPR chờ xử lý; plate, confidence, processed
anpr_runtime	Trạng thái runtime của ANPR và frame/camera

Lưu ý: các counter IN/OUT trong status_history cần được hiểu theo semantics của source hiện tại; không tự động coi chúng là số lượt xe thực tế nếu chưa có quy trình xác nhận gate.

7. ANPR hiện tại

ANPR service nằm tại Core/Python/ANPR/anpr_service.py. Service mở camera, chạy detector YOLO, crop biển số, gọi PlateOCR và đưa biển số hợp lệ vào bảng anpr_pending; đồng thời cập nhật anpr_runtime và ảnh latest_frame.jpg.

Các tham số hiện tại trong service gồm camera 0, 1280×720, 15 FPS, YOLO confidence 0.35, image size 640, OCR interval 0.7 s và cooldown lưu biển số 3 s.

Model bắt buộc để detector chạy là models/best.pt. Model này là tài nguyên dự án, khác với các model/cache tự tải của PaddleOCR.

ANPR không điều khiển slot/gate trực tiếp. ARM KIT vẫn là master; ANPR cung cấp biển số cho tầng PC.

8. Web Dashboard hiện tại

Flask app tại Core/Python/web/app.py đọc Core/Python/parking_history.db. Các endpoint chính: /, /api/status, /api/events, /api/anpr và /api/anpr/frame.

API /api/status dừng trạng thái 4 slot từ current_parking; API /api/anpr đọc anpr_runtime và anpr_pending. Điều này cho thấy ANPR đã có điểm nối vào Web ở tầng dữ liệu/API, nhưng chưa đồng nghĩa nghiệp vụ tự động “RFID + ANPR → gate → slot” đã hoàn tất.

9. EEPROM 24C16 và nguyên tắc bảo toàn

Không tự ý thay Record A/B hiện tại để nhét UID RFID. Trước tiên cần xác định vùng địa chỉ còn trống hoặc thiết kế record mới có version/CRC riêng. Nếu mở rộng storage, phải giữ khả năng boot đọc record cũ để không làm mất trạng thái parking hiện tại.

Nếu chỉ cần thử RFID, nên bắt đầu bằng danh sách UID trong RAM hoặc cấu hình cố định; sau khi giao tiếp ổn định mới đưa UID vào EEPROM.

10. RFID 13.56 MHz RS485 - kế hoạch tích hợp

Thông tin reader người dùng cung cấp: bản 13.56 MHz hỗ trợ Mifare 1 và có tùy chọn Wiegand26/34, RS232, RS485. Bản dự kiến nên chọn là 13.56 MHz + RS485.

Trước khi viết driver phải có protocol của đúng model: baud rate, data bits, parity, stop bits, địa chỉ/command nếu có, frame request/response, checksum/CRC, cách biểu diễn UID và trạng thái lỗi.

Sơ đồ logic đề xuất:

RFID Reader → RS485_1 → STM32 → UID authentication → gate permission → giữ nút GATE IN/OUT để mô phỏng → sau đó mới điều khiển barrier

Không dùng Hercules làm giao diện nghiệp vụ chính; nếu cần debug raw frame RS485 thì công cụ terminal chỉ là phương tiện kiểm tra phụ.

11. Quy trình phát triển RFID an toàn

Bước	Mục tiêu	Không làm ở bước này
1	Reader nhận thẻ và STM32 nhận UID	Không sửa gate/barrier
2	Chuẩn hóa UID và xác thực thẻ	Không thay cấu trúc EEPROM ngay
3	RFID hợp lệ → cho phép gate	Vẫn dùng nút mô phỏng
4	RFID + ANPR	Chưa điều khiển motor thật
5	RFID + ANPR + slot + DB	Chưa thay toàn bộ baseline
6	Driver motor + feedback	Chỉ triển khai sau khi logic ổn định
7	Nghiệm thu toàn hệ thống	Ghi log từng test case

12. Các test case cần có

- Thẻ hợp lệ → nhận UID → cho phép IN.
- Thẻ không hợp lệ → từ chối, không mở barrier.
- Bãi đầy → thẻ hợp lệ vẫn không được cấp quyền IN.
- Xe vào: gate permission → slot E→F → current_parking cập nhật.
- Xe ra: gate permission → slot F→E → exit_time cập nhật.
- ANPR đọc đúng biển số và tạo anpr_pending.
- ANPR đọc sai/không chắc chắn → không tự ý xác nhận xe.
- RFID và ANPR không khớp → đưa vào trạng thái cần xử lý, không tự động mở gate.
- Mất reader/RS485 → hệ thống parking cơ bản vẫn không bị treo.
- Reset STM32 → trạng thái slot hợp lệ được khôi phục từ EEPROM.

- Web không được tự quyết định IN/OUT; Web chỉ phản ánh dữ liệu đã được xác nhận.

13. Những phần không nên sửa khi chưa có lý do

- Không thay đổi semantics RAW/STAB chỉ để làm RFID để tích hợp.
- Không dùng lại PA9/PA10 cho RFID vì đang là RS232 PC.
- Không lấy PB3/PB4/PB5 cho RFID mà chưa kiểm tra 74HC165/595.
- Không nối motor trực tiếp vào DO của ARM KIT.
- Không dùng PE8/PE9 đồng thời làm input gate và output motor.
- Không đưa YOLO/PaddleOCR vào firmware STM32.
- Không đổi database path giữa PC Client và Web Dashboard.
- Không xóa best.pt nếu muốn chạy ANPR offline.
- Không biến một tính năng “có source” thành “đã nghiệm thu phần cứng” trong báo cáo.

14. Trạng thái bàn giao tại 07/09/2026

Thành phần	Trạng thái bàn giao
STM32 4 slot	Baseline firmware hiện tại
LCD/LED/Buzzer	Có
EEPROM 24C16	Có driver/storage logic
Gate state machine	Có phần mềm, chưa có motor barrier
Gate IN/OUT	Nút mô phỏng
RS232	Có, 115200 8N1
Python Client	Có
SQLite	Có; PC Client + Web dùng cùng DB
ANPR	Có camera/YOLO/OCR/service
Web Dashboard	Có Flask/API/UI
RFID	Chưa triển khai; chuẩn bị RS485 13.56 MHz
Motor barrier	Chưa triển khai
Cảm biến barrier thật	Chưa triển khai

Tài liệu này thay thế vai trò của hướng dẫn hiện trạng cũ; khi source thay đổi lớn, cần cập nhật lại tài liệu thay vì giữ các mô tả cũ không còn đúng.